ADA University

Nabiyev Rovshan 19794

Spring 2026 Intro to Embedded Systems (ENCE-3608 - 20291)

Date [May, 2026]

Lab Report

Lab Task 6: Two-Player Reaction Game

1. System Overview

This lab task implements a two-player reaction-time game using an Arduino Uno microcontroller paired with a Python-based host GUI running on a connected PC. The system measures each player's button-press latency in response to an audible buzzer signal, tracks wins across multiple rounds, and persists performance data to CSV files for post-session statistical analysis.

The complete system is composed of two tightly coupled components:

- Arduino (C++) — manages all real-time hardware interaction: button polling, timing, buzzer output, servo position, stepper motor movement, and serial communication with the host.
- Python GUI (Tkinter) — receives structured messages over UART, maintains game state and player names, writes CSV data, and renders statistics charts using Matplotlib.

2. Hardware Configuration

2.1 Pin Assignment and Component Roles

Pin	Component	Mode	Role
D3	Passive Buzzer	OUTPUT	Audible GO signal — 1000 Hz tone for 200 ms
D5	Player 2 button	INPUT	Active-LOW — reads LOW when button pressed
D6	Player 1 button	INPUT	Active-LOW — reads LOW when button pressed
D8	Stepper motor (IN4)	OUTPUT	28BYJ-48 coil driver — step sequencing
D9	Stepper motor (IN2)	OUTPUT	28BYJ-48 coil driver — step sequencing
D10	Stepper motor (IN3)	OUTPUT	28BYJ-48 coil driver — step sequencing
D11	Stepper motor (IN1)	OUTPUT	28BYJ-48 coil driver — step sequencing
D12	Servo motor	OUTPUT	Indicator — rotates to show which player won
A0	Floating ADC input	INPUT	Seeds randomSeed() for unpredictable delays

2.2 Button Wiring and INPUT_PULLUP Logic

Both player buttons use Arduino's internal pull-up resistors.

- The pin reads HIGH when the button is open — the pull-up holds the line to 5 V.
- The pin reads LOW when the button is pressed — the button shorts the pin to GND.

The firmware checks digitalRead(buttonPlayerN) == LOW to detect a press. This is the standard active-low configuration and eliminates the need for external resistors.

2.3 Stepper Motor (28BYJ-48)

The 28BYJ-48 is a 4-phase unipolar stepper driven in half-step mode via an ULN2003 driver board. The constructor argument ordering **Stepper myStepper(stepsPerRevolution, 11, 9, 10, 8)** matches the physical wiring of the four coil inputs to Arduino pins. With stepsPerRevolution = 2048 and a gear ratio of approximately 64:1, one full electrical revolution of 32 steps produces one mechanical output shaft revolution.

- Each point won moves the shaft ±50 steps (roughly ±8.8°), creating a visible tug-of-war effect on the indicator.
- The victorySpin() function rotates one full revolution (2048 steps) as a match-end celebration animation.
- resetGame() returns the shaft to its original position by stepping exactly -stepperPosition steps.

2.4 Servo Motor

A standard hobby servo on pin D12 is driven by the Arduino Servo library, which generates the required 50 Hz PWM signal using Timer 1. The servo position communicates the round winner visually:

moveServo() argument	Servo angle	Meaning
player = 1	0°	Player 1 won the round
(neutral)	90°	Between rounds / reset
player = 2	180°	Player 2 won the round

3. Firmware Operation

3.1 Program Flow

The firmware follows a deterministic loop with no RTOS or interrupt-driven timing. All game phases are sequential blocking operations:

setup() → Serial.begin(9600), servo neutral, pins configured, randomSeed()
loop() → resetGame() → playGame() → delay(5000) → repeat

Inside playGame(), the main while loop continues until either player reaches 3 wins. Each iteration is one round, consisting of three distinct phases:

3.2 Wait (False Start Detection)

A random delay between 1,000 ms and 20,000 ms is generated using random(1000, 20001). During this wait, the firmware polls both buttons in a tight loop. If either button is pressed before the delay expires, a false start is declared:

- The offending player loses the round; their opponent gains a point.
- updateStepper() shifts the stepper in the winner's direction.
- The string "FALSE START P1" or "FALSE START P2" is printed to the host over UART.
- The function returns immediately, ending the current match.

The random seed from analogRead(A0) ensures the wait duration is genuinely unpredictable between rounds, preventing players from timing their button presses.

3.3 Buzzer Signal

```
tone(buzzer, 1000);
delay(200);
noTone(buzzer);
```

The tone() function uses Timer 2 to generate a square wave without blocking the CPU. After 200 ms, startTime is captured with millis(), marking the exact reference point for reaction time measurement.

3.4 Reaction Measurement

Both players' button states are polled in a while loop until both have pressed. This ensures both reaction times are always recorded, even if one player presses much later:

```
while (!p1Pressed || !p2Pressed) {
  if (!p1Pressed && digitalRead(buttonPlayer1) == LOW) {
    reactionTime1 = millis() - startTime;
    p1Pressed = true;
  }
}
```

After both readings are captured, the player with the smaller reaction time wins. The winning time is sent to the host:

```
Serial.print("P1 WIN: ");
Serial.println(reactionTime1);
```

The servo rotates to the winner's side and the stepper advances 50 steps in the winner's direction. After a 1,500 ms pause (allowing players to see the result), the round loop continues.

3.5 Match End and Victory

When either score reaches 3, playGame() exits the while loop and prints the game winner:

```
Serial.println("GAME WINNER: P1");
```

victorySpin() then rotates the stepper a full revolution. After playGame() returns, loop() waits 5 seconds then calls resetGame() to return the stepper to centre before starting the next match.

4. Host GUI Operation

4.1 Serial Communication Protocol

The host GUI communicates with the Arduino over a virtual COM port at 9600 baud. A dedicated background thread (SerialManager._reader) reads bytes in 128-byte chunks and assembles them into newline-delimited strings, which are placed in a thread-safe queue. The main Tkinter event loop polls this queue every 30 ms via root.after() without blocking the UI.

The following message protocol is used:

Arduino message	Host action
GAME STARTS AUTOMATICALLY	Reset state to IDLE; show readiness message.
NEW ROUND	If IDLE/ENDED, call _begin_new_match(). Increment round counter. Update status label.
P1 WIN: <ms>	Parse reaction time. Award point to P1. Update scoreboard. Write CSV row.
P2 WIN: <ms>	Parse reaction time. Award point to P2. Update scoreboard. Write CSV row.
FALSE START P1/P2	Award point to non-offender. Write CSV. Set state to ENDED.
GAME WINNER: P1/P2	Announce match winner. Log victory to _victories.csv. Keep names for rematch.

4.2 Game State Machine

The host GUI maintains an explicit three-state machine to safely handle messages that arrive in any order

State	Meaning	Transitions to
IDLE	No match in progress. Awaiting NEW ROUND.	ACTIVE on NEW ROUND
ACTIVE	Match in progress. Recording rounds.	ENDED on score reaching 3 or false start
ENDED	Match complete. Names retained for rematch.	ACTIVE on next NEW ROUND (new session)

If a WIN or FALSE START arrives while in IDLE or ENDED state, _begin_new_match() is called defensively to avoid data loss.

4.3 Player Name Persistence

Names entered in the GUI are "armed" and applied at the start of the next match. Once applied, they persist in self.p1_name and self.p2_name for the lifetime of the session. Crucially, names are NOT cleared after a match ends or after a false start, so rematches between the same players require no re-entry. New names take effect only when "Arm for next match" is clicked again and a new NEW ROUND message arrives.

4.4 Data Persistence

- Per-player files one row per round, recording timestamp, session ID, round number, opponent name, reaction time, won flag, and false start flag.
- Victory log one row per completed match, recording winner, loser, session ID, and final score string

5. Statistics Visualisation

The Statistics tab provides three Matplotlib plots selectable via a dropdown:

- Reaction time over sessions plots average and best winning reaction time per session ID for the selected player. Helps identify improvement or fatigue trends across multiple play sessions.
- Win rate by opponent bar chart showing percentage of rounds won against each unique opponent. Useful for identifying consistent strengths or weaknesses against specific competitors.
- Head-to-head reaction times scatter/line plot of all winning reaction times against a chosen opponent in chronological order, with a horizontal mean line. Reveals consistency and outliers within a rivalry.

All three plots exclude false-start rounds from reaction time calculations to avoid corrupting timing statistics with invalid data.

6. Timing Accuracy and Known Limitations

6.1 Reaction Time Resolution

millis() on the Arduino Uno has a resolution of approximately 1 ms and is driven by Timer 0 at a 64-prescaler on the 16 MHz system clock. This gives a theoretical precision of ±1 ms. In practice, the main polling loop adds sub-millisecond latency between the actual button press and the millis() capture, so the true accuracy is approximately ±2–4 ms acceptable for a human reaction time game where typical values range from 150–400 ms.

6.2 Serial Latency

At 9600 baud, each byte takes approximately 1.04 ms to transmit. A typical message such as "P1 WIN: 342\n" takes roughly 12.5 ms to arrive fully at the host. This delay is purely cosmetic and does not affect reaction time measurement, which occurs entirely on the Arduino before any serial transmission.

6.3 Simultaneous Button Presses

The firmware polls both buttons in a simple sequential if structure within the same loop iteration. If both players press within the same 1 ms polling window, the player whose if branch is evaluated first P1 will

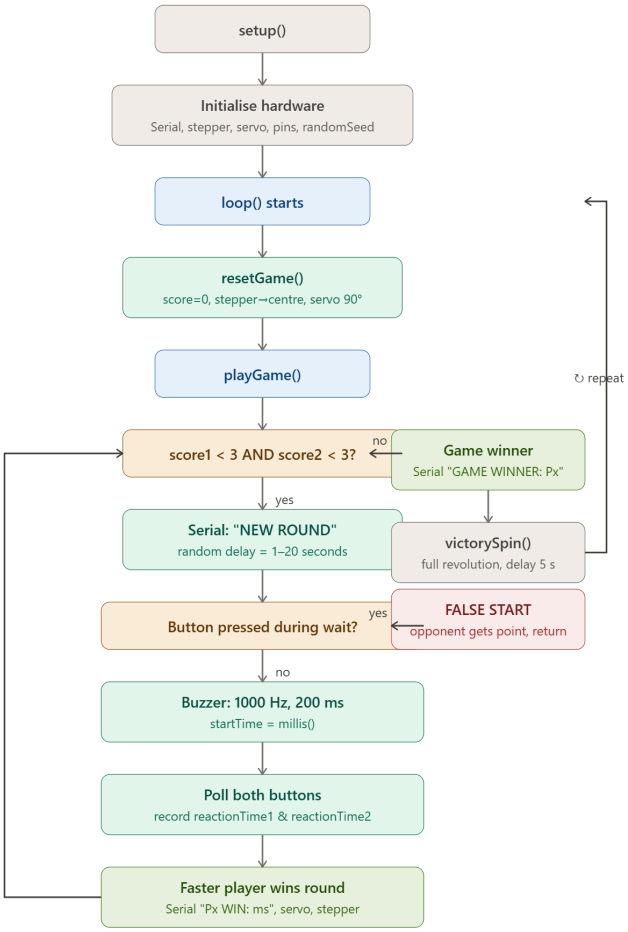
record a marginally earlier timestamp. This is an inherent limitation of the polling approach and could be improved with hardware interrupts on INT0/INT1 pins in a future revision.

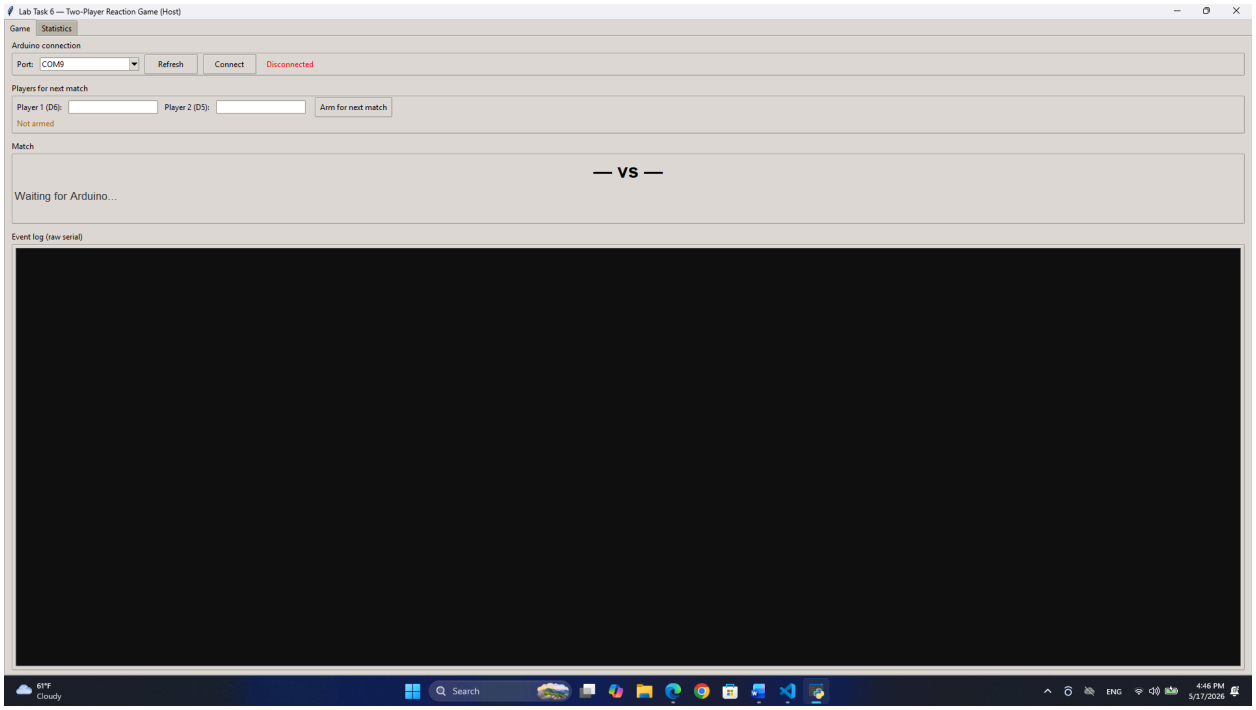
6.4 Stepper Blocking

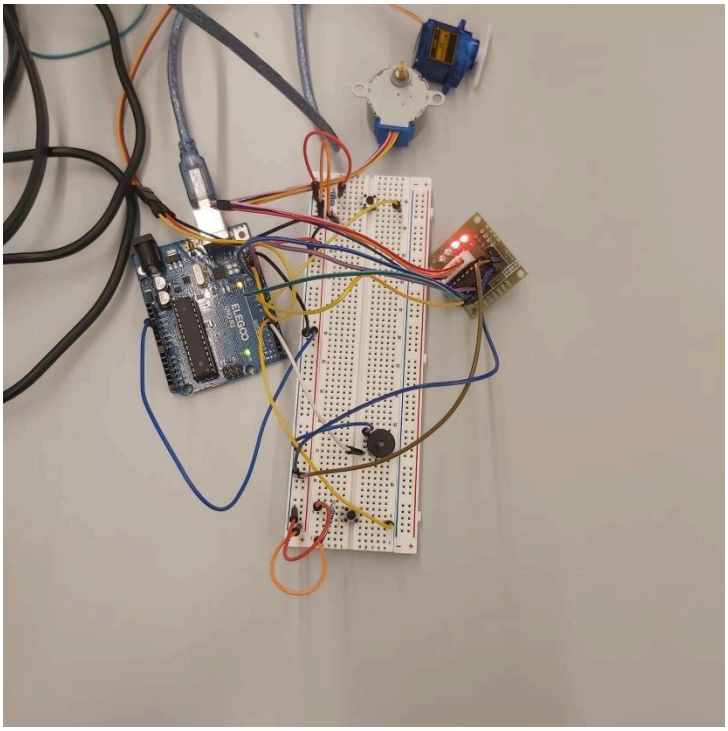
The Stepper library's step() function is synchronous and blocking. Each updateStepper() call (50 steps at 10 RPM) takes approximately 300 ms. During this time, button inputs and serial output are paused. This is mitigated by the 1,500 ms post-round delay, which accommodates the stepper movement without affecting gameplay.

7. GUI , Timing, and Project's pictures

VIDEO YOUTUBE LINK
https://youtube.com/shorts/mzLCyhH6TaY?feature=share




Lab Task 6 — Two-Player Reaction Game (Host)
Game
Statistics
Arduino connection
Port: COM9
Refresh
Connect
Disconnected
Players for next match
Player 1 (D6):
Player 2 (D5):
Arm for next match
Not armed
Match
— vs —
Waiting for Arduino...
Event log (raw serial)
61°F
Cloudy
Search
ENG
4:46 PM
5/17/2026


ELEGOO

8. Conclusion

The Lab Task 6 solution demonstrates a clean separation between real-time hardware control and persistent data management with user-facing visualisation. The Arduino handles all timing-critical operations locally without relying on host round-trip latency, while the host provides a rich interface for player management, data logging, and statistical insight. Key design decisions such as INPUT_PULLUP button wiring, randomised wait durations, explicit game state management on the host, and CSV-based persistence combine to produce a robust, extendable reaction-time gaming platform.

Lab Task 7: Multi-Component Security System

1. System Overview

Lab Task 7 implements a multi-input access-control system on Arduino Uno. The system combines three distinct input devices a 4×4 membrane keypad, an NEC infrared (IR) remote receiver, and an SPI-based RFID-RC522 module with two LED indicators and a host-side Python GUI that logs RFID tag events to a SQLite database.

The firmware enforces a strict three-state security flow: a user first sets a 4-digit code on the keypad to lock the system; the correct code entered via the IR remote unlocks it; and only in the unlocked state does the RFID reader become active, logging any card or fob presented to it. Each state is signaled visually by the LEDs and reported over UART to the host GUI.

2. Hardware Configuration

2.1 Pin Assignment

Pin(s)	Peripheral	Direction	Function
A0–A3	Keypad rows	Input (pull-up via Keypad lib)	Scanned to detect key presses
D2–D5	Keypad columns	Output (driven low for scan)	Column drive lines for key matrix
D6	IR receiver (TSOP)	Input	Demodulated 38 kHz NEC signal in
D7	Red LED	Output	Locked / waiting indicator
D8	Green LED	Output	Unlocked indicator
D9	RFID RST	Output	RC522 hard reset line
D10	RFID SS/SDA	Output (CS)	SPI chip-select for RC522
D11	RFID MOSI	Output (SPI)	Master-out data to RC522
D12	RFID MISO	Input (SPI)	Master-in data from RC522
D13	RFID SCK	Output (SPI)	SPI clock to RC522
A0 (ADC)	Floating ADC	Input (ADC)	Not used by Task 7 firmware

A critical build-time decision is the macro DISABLE_LED_FEEDBACK passed to IrReceiver.begin(). By default, the IRremote v4 library toggles pin D13 as a feedback LED whenever an IR frame is received. Because D13 is the hardware SPI clock shared with the RC522, allowing the library to drive it would corrupt any in-progress SPI transaction. The macro suppresses this behavior entirely.

2.2 RFID-RC522 and SPI

The RC522 is a 13.56 MHz RFID front-end that communicates over SPI at up to 10 MHz It is initialized in setup() with three explicit steps beyond the basic PCD_Init():

- Antenna gain is set to maximum (RxGain_max = 0x07 << 4, giving approximately 48 dB) using PCD_SetAntennaGain(). The default gain of ~33 dB is insufficient for many clone chips commonly used in lab kits.
- PCD_AntennaOn() is called explicitly to re-energise the RF field after the gain register write, which can transiently disable the antenna on some silicon revisions.
- The firmware reads the VersionReg register (expected 0x91 or 0x92 for a genuine NXP chip) over SPI and prints the result as INFO,rfid_version_0xXX. A response of 0x00 or 0xFF indicates a wiring or power fault and is flagged with an additional INFO,rfid_NOT_RESPONDING message, allowing the user to diagnose the issue without an oscilloscope.

2.3 IR Receiver and NEC Protocol

The TSOP38238 demodulates the 38 kHz carrier and outputs a logic-level NEC-encoded bitstream to D6. The IRremote v4 library decodes this into a 32-bit raw value. Decoding is enabled at compile time with #define DECODE_NEC placed before the library header, which selects only the NEC decoder and reduces code-size compared to enabling all protocols.

The decoded 32-bit value is matched against a look-up table (IR_TABLE) stored in program flash (PROGMEM) using memcpy_P to avoid copying the entire table into SRAM. Each entry maps one remote button to a character ('0'–'9', '*', '#'). The optional DEBUG_IR flag prints every raw IR code over serial, allowing the operator to remap the table for any NEC-compatible remote without modifying library internals.

Repeat frames are identified by the IRDATA_FLAGS_IS_REPEAT flag and are ignored, preventing a single long press from entering multiple digits.

2.4 Keypad

The Keypad library implements a software key-matrix scan across the 4×4 grid. Each call to keypad.getKey() drives one column low at a time and reads all four row pins to detect which key, if any, is pressed. The library de-bounces internally and returns a single character per press event. The analog pins A0–A3 are used as digital I/O via their alternative digital pin numbers; no ADC functionality is required.

3. Firmware State Machine

3.1 States and Transitions

The firmware implements an explicit three-state machine using an enum SystemState with values ST_WAITING, ST_LOCKED, and ST_UNLOCKED. All transitions are performed through the enterState() helper, which atomically resets the code input buffer, clears the active code if returning to WAITING, and broadcasts the new state over UART.

State	LED pattern	Active inputs	Transition out
ST_WAITING	Red blinks at 1 Hz, green off	Keypad only	4-digit code entered on keypad → ST_LOCKED
ST_LOCKED	Red solid, green off	IR receiver only	Correct 4-digit code via IR remote → ST_UNLOCKED
ST_UNLOCKED	Red off, green solid	RFID + keypad	Keypad '*' pressed → ST_WAITING

The strict separation of active inputs per state provides a layered security model: physical access sets the code, remote IR entry unlocks, and RFID logging is only available to an authenticated user. The RFID reader is explicitly gated by an early return inside rfidPoll() when state != ST_UNLOCKED, ensuring the SPI bus is not unnecessarily exercised while locked.

3.2 Code Buffer and Matching

Two-character arrays manage PIN entry. codeBuf accumulates digits as they arrive from either input device; activeCode holds the code that was set during the most recent WAITING phase. Both buffers are null-terminated C strings.

The handleDigit() function processes one character at a time from either input source or enforces the following rules:

- '*' : resets codeBuf to empty and announces INFO,cancel over serial.
- '0'–'9': appended to codeBuf if fewer than CODE_LEN (4) digits have been entered. Each digit triggers INFO,digit_kp or INFO,digit_ir depending on source.
- '#' : triggers the same logic as auto-confirm but only if 4 digits are already present, allowing the user to press # redundantly without causing an error.
- Auto-confirm at 4 digits: if state is ST_WAITING and the source is the keypad, storeActiveCode() copies codeBuf into activeCode and enterState(ST_LOCKED) is called. If state is ST_LOCKED and the source is the IR remote, codeMatches() is called; a match transitions to ST_UNLOCKED, a mismatch announces INFO,wrong_code and resets the buffer.

The deliberate asymmetry keypad sets, IR unlocks means the IR remote cannot be used to set a new code, and the keypad cannot be used to unlock the system. This prevents an attacker who has intercepted the IR signal from simply re-entering the code via keypad.

4. LED Patterns and Visual Feedback

All LED state management is centralised in updateLeds(), called every iteration of loop(). The function uses millis() for non-blocking timing, preserving responsiveness to all three input devices.

Condition	Red LED	Green LED	Purpose
ST_WAITING	Blinks every 500 ms	Off	Signals the system is unarmed and awaiting code entry

Condition	Red LED	Green LED	Purpose
ST_LOCKED	Solid on	Off	System is armed; only IR remote accepted
ST_UNLOCKED	Off	Solid on	System is open; RFID active
RFID card read (flash)	Alternates 80 ms	Alternates 80 ms	Brief dual-flash confirms successful tag scan (~3 cycles)

The RFID double-flash is implemented via a flashUntilMs timestamp. When a card is successfully read, flashUntilMs is set to millis() + 480 ms. On each updateLeds() call, if the current time is within this window, both LEDs alternate at 80 ms intervals, overriding the normal state pattern. Once the window expires, the flag is cleared and normal LED behaviour resumes. This approach requires no additional timers or interrupts.

5. Serial Communication Protocol

The firmware communicates with the host at 115200 baud using a simple comma-separated text protocol. Each message is a single newline-terminated line. The higher baud rate reduces per-character latency to approximately 87 µs, which is important for the diagnostic IR debug output where multiple lines may be generated per remote button press.

Message format	Direction	Meaning
STATE,WAITING	Arduino → PC	System entered WAITING state
STATE,LOCKED	Arduino → PC	System entered LOCKED state
STATE,UNLOCKED	Arduino → PC	System entered UNLOCKED state
TAG,<UID_HEX>	Arduino → PC	RFID card scanned; UID as hex string
INFO,boot	Arduino → PC	Firmware initialization complete
INFO,rfid_version_0xXX	Arduino → PC	RC522 silicon version register value
INFO,rfid_OK	Arduino → PC	RC522 responding correctly
INFO,rfid_NOT_RESPONDING	Arduino → PC	RC522 wiring/power fault detected
INFO,code_set_XXXX	Arduino → PC	Active 4-digit code has been set
INFO,digit_kp / digit_ir	Arduino → PC	Digit received from keypad or IR remote
INFO,wrong_code	Arduino → PC	IR code entry did not match active code
INFO,cancel	Arduino → PC	Code entry cancelled via '*'
INFO,ir_raw_0xXXXXXXXX	Arduino → PC	Raw 32-bit IR code
INFO,rfid_card_detected	Arduino → PC	Card present signal asserted
INFO,rfid_read_failed	Arduino → PC	Card present but UID read unsuccessful

6. Host GUI Operation (tag_logger.py)

6.1 Architecture

The Python GUI is built on PyQt6 with a strict two-thread architecture. All serial I/O is performed in a QThread subclass that reads lines from the COM port using a blocking readline() call with a 0.5-second timeout. Parsed events are delivered to the GUI thread exclusively through PyQt6 signals, ensuring that SQLite writes and widget updates always occur on the main thread and avoiding race conditions.

6.2 SQLite Database Schema

Tag events are persisted in a local SQLite file using a single table:

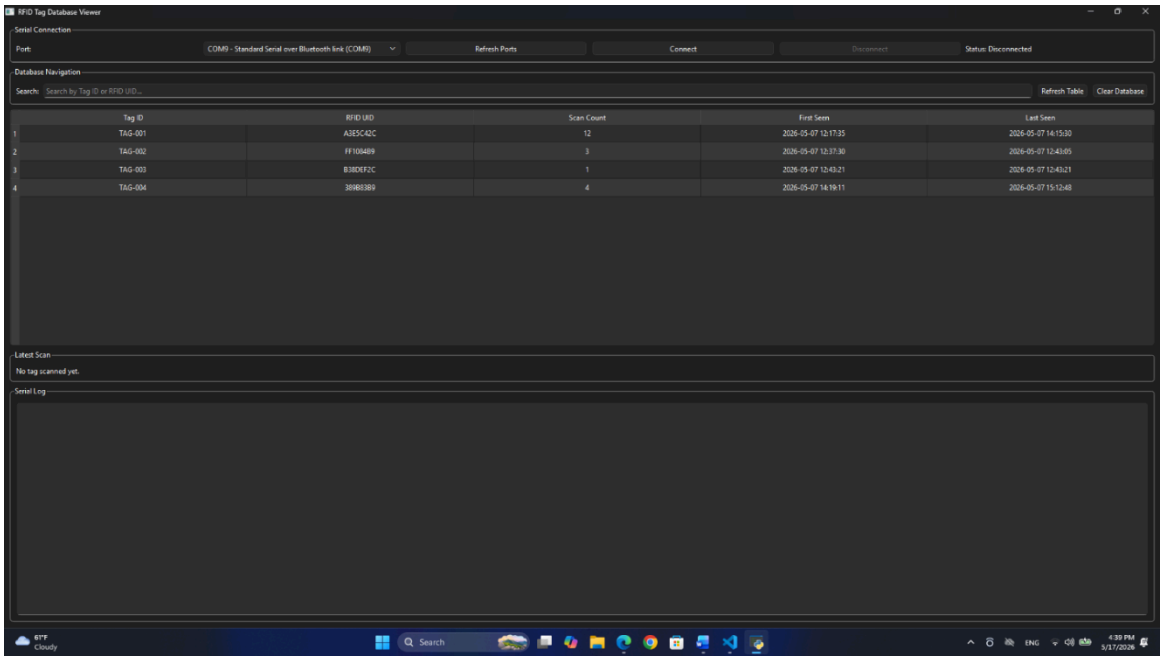
```
CREATE TABLE tags (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    uid         TEXT    NOT NULL UNIQUE,
    label       TEXT    DEFAULT '',
    scan_count  INTEGER NOT NULL DEFAULT 1,
    first_seen  TEXT    NOT NULL,
    last_seen   TEXT    NOT NULL
);
```

The uid column carries a UNIQUE constraint, so each physical card generates exactly one row regardless of how many times it is scanned. The upsert_scan() method uses a SELECT-then-INSERT-or-UPDATE pattern: if the UID is absent a new row is created; if it already exists, scan_count is incremented and last_seen is updated. An index on uid (idx_tags_uid) ensures O(log n) lookup even for large tag databases. ISO-8601 strings (YYYY-MM-DDTHH:MM:SS) are used for timestamps, making the data directly sortable as text without any date-parsing overhead.

6.3 GUI Features

The main window is divided by a resizable splitter into a tag table and a serial log.

- Sortable table all six columns support ascending/descending sort via the QTableWidget header. Sorting is temporarily disabled during batch refresh (blockSignals + setSortingEnabled(False)) to prevent itemChanged signals from firing on programmatic updates.
- Inline label editing only the Label column is editable. Double-clicking or single-clicking a selected label cell opens an inline editor; on commit, _on_item_changed() writes the new label to SQLite immediately. All other columns have ItemIsEditable cleared.
- Live state indicator a colour-coded pill label in the top bar updates on every STATE message: amber for WAITING, red for LOCKED, green for UNLOCKED, dark grey for DISCONNECTED.
- Delete with confirmation selected rows are removed from both the table widget and the SQLite database after a Yes/No confirmation dialog.
- Scrolling serial log a QPlainTextEdit limited to 500 blocks (setMaximumBlockCount) displays every raw line received from the Arduino with a HH:MM:SS timestamp prefix. This buffer prevents unbounded memory growth during long sessions.

- 

6.4 Connection Lifecycle

Clicking Connect opens the selected serial port in the SerialReader thread and wires all signals. Clicking Disconnect calls reader.stop() and reader.wait(1500) to give the thread up to 1.5 seconds to exit cleanly before the port is released. If the thread terminates unexpectedly, the finished signal triggers _on_reader_finished() which resets the UI to the disconnected state. The closeEvent handler repeats this teardown sequence so the port is always released on application exit.

7. Timing and Performance Considerations

7.1 loop() Latency

Each iteration of loop() calls four subsystems sequentially: keypad.getKey(), IrReceiver.decode(), rfidPoll(), and updateLeds(). None of these blocks for significant time during normal operation. The dominant latency source is rfid.PICC_IsNewCardPresent(), which sends a short RF command and waits for a response; this typically completes in under 5 ms. The overall loop period is therefore well under 10 ms, giving a keypad poll rate above 100 Hz and an IR decode latency well within human perception thresholds.

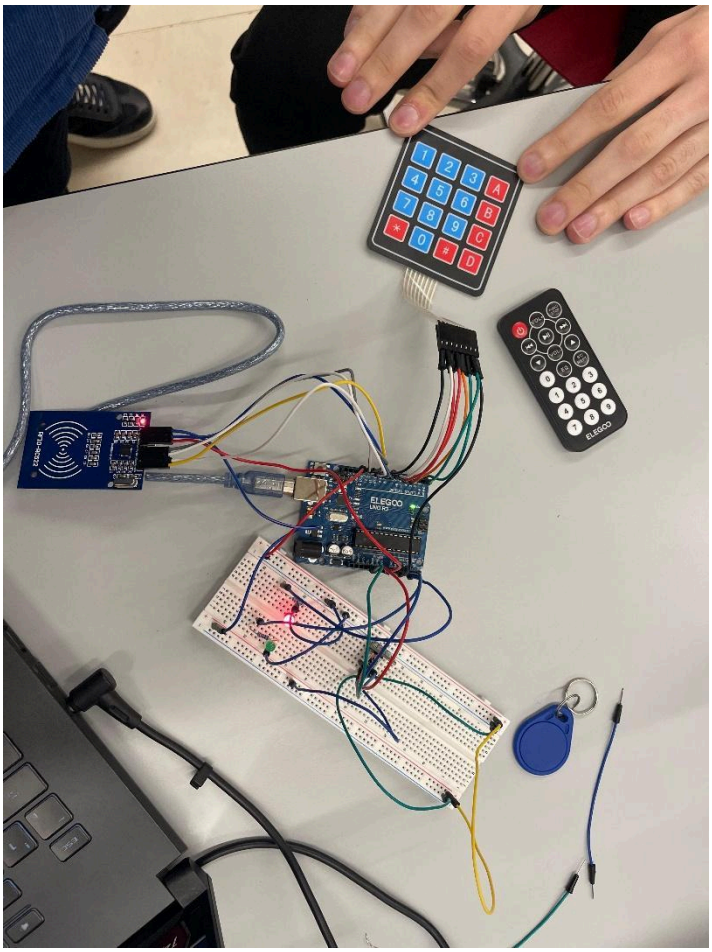
7.2 IR Decoding

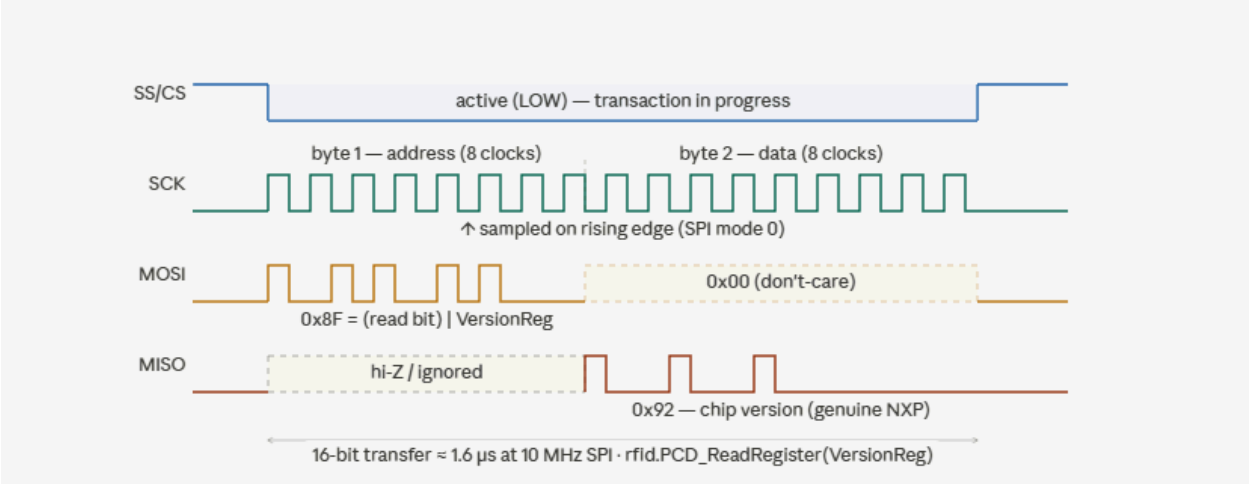
NEC frames consist of a 9 ms leading burst, a 4.5 ms space, and 32 data bits each encoded as a 0.56 ms pulse plus a 0.56 ms (logic 0) or 1.69 ms space. The total frame duration is approximately 67.5 ms. The IRremote library captures this via a hardware timer interrupt, so the CPU is free to run loop() during reception. IrReceiver.decode() returns immediately with a result flag; if no complete frame is available, it returns false and loop() continues without delay.

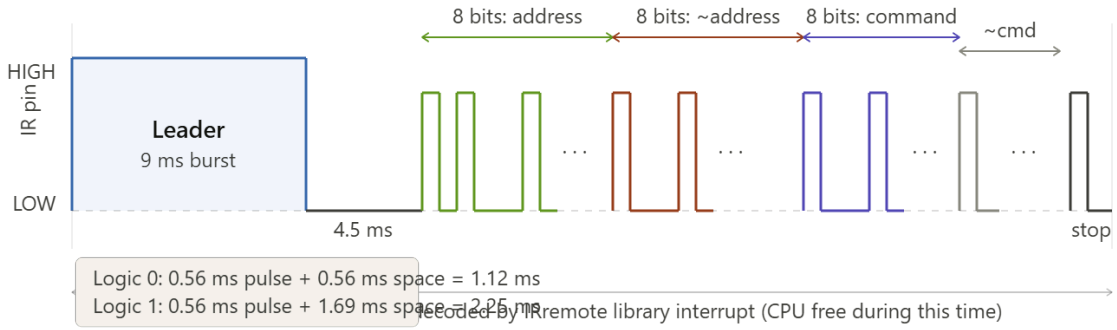
7.3 RFID Anti-Collision

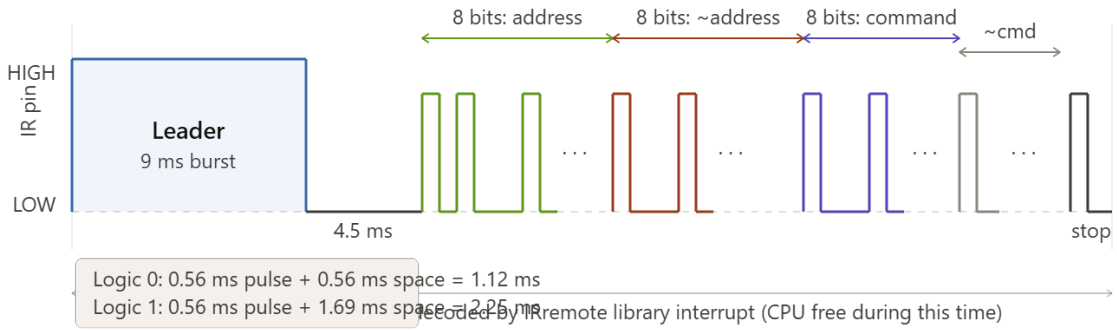
The PICC_IsNewCardPresent() and PICC_ReadCardSerial() call sequence implements the ISO/IEC 14443-3 Type A anti-collision protocol at the library level. After a successful read, PICC_HaltA() sends a HALT command to put the card into the HALT state, and PCD_StopCrypto1() resets the RC522's internal crypto engine. Without these calls, the same card would trigger repeated TAG messages on every loop iteration until moved out of the RF field.

8. System Flow Summary and Project Images









9. Conclusion

Lab Task 7 demonstrates a layered multi-peripheral embedded security system. The three-state machine enforces a clear separation between code-setting (keypad), code-entry (IR remote), and authenticated use (RFID), with each state activating only the relevant input hardware. The firmware avoids blocking delays throughout, using millis()-based timers and library

interrupt-driven drivers so all peripherals remain responsive simultaneously. On the host side, the PyQt6 GUI cleanly separates serial I/O (worker thread) from database writes and UI updates (main thread), providing a robust and extensible tag logging platform backed by SQLite persistence.